

Conversion operators, Function objects

RPi Pico is designed to use RP2040 microcontroller. RP2040 allows GPIO input and output to drive inverted values, so that 0 on input reads as 1 when the state of the GPIO pin is read. This mechanism works separately for input and output. By default the GPIOs are in normal state. By calling:

```
gpio_set_outover(gpio_pin_number, GPIO_OVERRIDE_INVERT);  
gpio_set_inover(gpio_pin_number, GPIO_OVERRIDE_INVERT);
```

You can set either output or input to be inverted.

Implement a class that abstracts DigitalIO Pins and a program that uses the class.

The class must have the following interface:

```
class GPIOPin {  
public:  
    GPIOPin(int pin, bool input = true, bool pullup = true, bool invert = false);  
    GPIOPin(const GPIOPin &) = delete;  
    bool read();  
    bool operator()(); // returns the state of the pin  
    void write(bool value);  
    void operator()(bool); // this writes a value to the pin  
    operator int(); // this returns the GPIO pin number  
private:  
    // add these as needed  
};
```

Callouts for the GPIOPin class interface:

- Constructor parameters:**
 - `bool input`: true → input, false → output
 - `bool pullup`: true → enable pullup, false → no pullup. Has no effect with output pin.
 - `bool invert`: true → invert, false → normal
- `GPIOPin(const GPIOPin &) = delete;`**: Makes the object non-copyable
- `bool read();`**: Sets the state of pin. Has no effect when pin is configured as an input.

The development board has three buttons and three LEDs.

The buttons are connected to the following pins:

- SW0 → Pin 9
- SW1 → Pin 8
- SW2 → Pin 7

All three buttons require that pullups are enabled. When using pullups a button reads zero when it is pressed and one when not pressed. This can be changed by enabling the inverter on the IO-pins.

The onboard LEDs are connected to:

- LED1 – Pin 22
- LED2 – Pin 21
- LED3 – Pin 20

Implement a program that creates six GPIOPin objects, one for each on-board button, and one for each of the the onboard LEDS.

Create the objects locally (do not use new!). For example:

```
GPIOPin sw0(9, true, true, false);
```

The program monitors the state of the pins at 10 ms intervals and switches LEDs on when the corresponding button is pressed.

When SW2 is pressed LED1 is on. SW1 switches on LED2 and SW0 switches on LED3.

The LEDs must stay on as long as the controlling button is pressed. When the button is released, the LED is switched off after one second. It must be possible to release buttons at different times.

For example:

